Unix & Shell Programming

Module 4 – Utility Commands

Question & Answer Notes

BCA Semester 6 · MAKAUT

Q1. The `cal` and `date` Commands

cal – Display a Calendar

The `cal` command prints a calendar on the terminal.

Syntax: `cal [options] [[month] year]`

```
cal                     # show current month's calendar
cal 2025                # show full calendar for year 2025
cal 6 2025              # show June 2025
cal -3                  # show previous, current, and next month
cal -y                  # show all 12 months of the current year
cal -j                  # show Julian dates (day of year: 1-365)
cal -m                  # week starts on Monday (default is
   Sunday)
cal -s                  # week starts on Sunday (explicit)
```

Example output of `cal 6 2025`:

```
      June 2025
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30
```

cal options summary:

Option	Description
`cal`	Current month
`cal YEAR`	All months of that year
`cal MM YYYY`	Specific month and year
`cal -3`	Previous, current, next month
`cal -y`	All 12 months of current year
`cal -j`	Julian day numbers
`cal -m`	Start week on Monday

date – Display or Set the System Date and Time

The `date` command shows the current date and time. With root access, it can also set the system clock.

Syntax: `date [options] [+FORMAT]`

```
date                              # show full current date and time
# Output: Mon Jun  9 10:30:45 IST 2025

date +"%d/%m/%Y"                  # Output: 09/06/2025
date +"%H:%M:%S"                  # Output: 10:30:45
date +"%A, %B %d %Y"              # Output: Monday, June 09 2025
date +"%Y-%m-%d"                  # Output: 2025-06-09  (ISO format
   )
date +"%I:%M %p"                  # Output: 10:30 AM    (12-hour
   format)
date +"%s"                        # Output: Unix timestamp (seconds
    since 1970)
```

Useful format codes for `date`:

Code	Meaning	Code	Meaning
%d	Day (01-31)	%H	Hour 24h (00-23)
%m	Month (01-12)	%I	Hour 12h (01-12)
%Y	Year (4-digit)	%M	Minutes (00-59)
%y	Year (2-digit)	%S	Seconds (00-59)
%A	Full weekday name	%p	AM or PM
%a	Short weekday	%s	Unix timestamp
%B	Full month name	%Z	Timezone name
%b	Short month name	%j	Day of year (001-366)

```
date -d "2025-01-15"              # show date for a specific date
   string
date -d "next Monday"             # date of next Monday
date -d "3 days ago"              # date 3 days before today
date -d "+2 weeks"                # date 2 weeks from now
date -d @1700000000               # convert Unix timestamp to
   readable date

# Setting the date (requires root):
date -s "2025-06-09 10:30:00"
```

Practical tip: Use `date` in scripts to create timestamped log files:
`logfile="backup_$(date +%Y%m%d_%H%M%S).log"`
This creates a filename like `backup_20250609_103045.log`

Q2. The pr and who Commands

pr – Format Files for Printing

The pr command prepares a text file for printing. It adds headers, page numbers, and formats the output into pages.

Syntax: pr [options] file

```
pr file.txt                    # format file with header (
   filename, date, page no.)
pr -n file.txt                 # add line numbers to the output
pr -d file.txt                 # double-space the output
pr -h "My Report" file.txt     # custom header text instead of
   filename
pr -l 40 file.txt              # set page length to 40 lines
pr -w 60 file.txt              # set page width to 60 characters
pr -t file.txt                 # suppress header and footer (
   title-less)
pr -2 file.txt                 # format output into 2 columns
pr -3 file.txt                 # format output into 3 columns
pr -m file1 file2              # merge files side by side (like
   paste)
```

Example header added by pr:

```
2025-06-09 10:30          file.txt          Page 1

[content starts here]
```

pr options summary:

Option	Description
pr -n	Number each line
pr -d	Double space lines
pr -h STR	Use STR as the page header
pr -l N	Set page length to N lines (default 66)
pr -w N	Set page width to N characters (default 72)
pr -t	No header or footer
pr -N	Format into N columns (e.g., -2, -3)
pr -m	Merge and print files side by side

who – Show Logged-in Users

The who command displays information about users currently logged into the system.

Syntax: who [options]

```
who                    # show all currently logged-in users
# Output format: username  terminal  login-time  (IP/host)
# Example:
```

```
# shadow    pts/0    2025-06-09 09:15 (:0)
# root      pts/1    2025-06-09 10:00 (192.168.1.10)
```

who options:

```
who -b                    # show last system boot time
who -q                    # show only usernames and count
who -H                    # show column headers
who -r                    # show current run level
who am i                  # show info about your own terminal
   session
whoami                    # show your own username (simpler)
w                         # extended who -- shows what each user
   is doing
last                      # show login/logout history of all users
```

Example outputs:

```
who -H
# NAME        LINE          TIME              COMMENT
# shadow      pts/0         2025-06-09 09:15 (:0)

who -q
# shadow root
# # users=2

whoami
# shadow

who am i
# shadow    pts/0    2025-06-09 09:15 (:0)
```

Difference: `whoami` tells you your username. `who am i` tells you your full session info. `who` lists all users. `w` shows what each user is currently doing.

Q3. The bc Command – Basic Calculator

`bc` stands for **Basic Calculator**. It is an arbitrary-precision calculator language built into UNIX. It supports integers, decimals, and even programming constructs.

Syntax: `bc [options]`

Interactive mode – just type `bc` and press Enter:

```
bc
# Now type expressions:
5 + 3             # Output: 8
10 - 4            # Output: 6
6 * 7             # Output: 42
20 / 4            # Output: 5
2 ^ 10            # Output: 1024  (power/exponent)
```

```
17 % 5          # Output: 2    (modulus/remainder)
quit            # exit bc
```

Decimal precision with `scale`:

By default, `bc` does integer division. Set `scale` for decimal results.

```
bc
scale=2
22 / 7          # Output: 3.14
scale=5
22 / 7          # Output: 3.14285
sqrt(2)         # Output: 1.41421  (square root)
```

Using bc non-interactively (via echo and pipe):

```
echo "5 + 3" | bc                  # Output: 8
echo "scale=2; 22/7" | bc         # Output: 3.14
echo "2^10" | bc                   # Output: 1024
echo "sqrt(144)" | bc -l           # Output: 12.00000...
echo "scale=4; sqrt(2)" | bc -l    # Output: 1.4142
```

bc in shell scripts:

```
result=$(echo "scale=2; 100 / 3" | bc)
echo "Result is: $result"
# Output: Result is: 33.33

a=15; b=4
sum=$(echo "$a + $b" | bc)
echo "Sum = $sum"     # Output: Sum = 19
```

bc with -l (math library):

The `-l` flag loads the math library with extra functions.

```
echo "scale=6; s(3.14159)" | bc -l   # sin() function
echo "scale=6; c(0)" | bc -l         # cos() function
echo "scale=6; l(1)" | bc -l         # natural log (ln)
echo "scale=6; e(1)" | bc -l         # e^1 = 2.718281
```

bc operators summary:

Operator	Meaning	Operator	Meaning
`+`	Addition	`^`	Power
`-`	Subtraction	`%`	Modulus
`*`	Multiplication	`sqrt(n)`	Square root
`/`	Division	`scale=N`	Decimal places

Why use bc? The shell itself only does integer arithmetic. For decimal calculations in scripts, always use `bc`. Example: `echo "scale=2; $price * 1.18" | bc` (add 18% GST).

Q4. The `echo` Command

The `echo` command prints text or variable values to the screen (standard output).

Syntax: `echo [options] [string]`

Basic usage:

```
echo "Hello, World!"           # Output: Hello, World!
echo Hello World               # Output: Hello World  (quotes
   optional)
echo ""                        # print a blank line
echo "Today is $(date)"        # embed command output in string
```

Printing variables:

```
name="Shadow"
echo "My name is $name"        # Output: My name is Shadow
echo "My name is ${name}!"     # Output: My name is Shadow!
echo $PATH                     # print PATH environment variable
echo $HOME                     # print home directory
echo $USER                     # print current username
```

echo options:

Option	Description
`echo -n`	Do NOT print newline at end (cursor stays on same line)
`echo -e`	Enable interpretation of escape sequences (backslash codes)
`echo -E`	Disable escape sequences (default behavior)

Escape sequences with `echo -e`:

Code	Meaning	Code	Meaning
\n	Newline (new line)	\t	Tab
\r	Carriage return	\a	Bell (beep)
\b	Backspace	\\	Literal backslash

```
echo -e "Line1\nLine2\nLine3"
# Output:
# Line1
# Line2
# Line3
```

```
echo -e "Name:\tShadow"
# Output: Name:    Shadow

echo -e "Hello\tWorld\n---\nBye"
# Output:
# Hello    World
# ---
# Bye

echo -n "Enter name: "        # no newline, cursor stays at
   same line
```

echo with redirection (writing to files):

```
echo "Hello" > greet.txt       # write to file (overwrite)
echo "World" >> greet.txt      # append to file
echo -e "Line1\nLine2" > file.txt  # write multiple lines to
   file
```

echo with colors (using ANSI codes):

```
echo -e "\033[31mThis is Red Text\033[0m"
echo -e "\033[32mThis is Green Text\033[0m"
echo -e "\033[1mThis is Bold Text\033[0m"
```

Common use in scripts: `echo` is used to display messages, print variable values, write output to files, and show progress in shell scripts. It is one of the most frequently used UNIX commands.

Q5. `zip`, `unzip`, and `gzip` Commands

These commands are used to **compress and decompress** files to save disk space.

zip – Create a ZIP archive

Syntax: `zip [options] archive.zip file(s)`

```
zip myarchive.zip file1.txt file2.txt    # zip multiple files
zip myarchive.zip *.txt                   # zip all .txt files
zip -r myarchive.zip myfolder/           # zip entire directory
   (-r = recursive)
zip -r project.zip src/ docs/ README     # zip multiple items

zip -e secret.zip file.txt              # create encrypted (
   password) zip
zip -9 myarchive.zip file.txt           # maximum compression (
   level 9)
zip -0 myarchive.zip file.txt           # no compression (just
   store)
```

```
zip -v myarchive.zip file.txt           # verbose output

zip -u myarchive.zip newfile.txt        # update (add/replace)
    file in zip
zip -d myarchive.zip file1.txt          # delete a file from zip
zip -l myarchive.zip                    # list contents of zip (
    same as unzip -l)
```

unzip – Extract a ZIP archive

Syntax: `unzip [options] archive.zip`

```
unzip myarchive.zip                     # extract all files to
    current directory
unzip myarchive.zip -d /home/shadow/    # extract to a specific
    directory
unzip -l myarchive.zip                  # list contents without
    extracting
unzip -v myarchive.zip                  # verbose list with
    sizes and CRC
unzip -t myarchive.zip                  # test zip integrity
unzip myarchive.zip file1.txt           # extract only one
    specific file
unzip -o myarchive.zip                  # overwrite existing
    files without asking
unzip -n myarchive.zip                  # never overwrite
    existing files
unzip -q myarchive.zip                  # quiet mode (no output)
```

gzip – GNU Zip (compress a single file)

`gzip` compresses a single file and replaces it with a `.gz` file. Unlike `zip`, it does not bundle multiple files together.

Syntax: `gzip [options] file`

```
gzip file.txt                   # compress file.txt -> file.txt.
    gz (original deleted)
gzip -k file.txt                # keep original file (don't
    delete it)
gzip -d file.txt.gz             # decompress (same as gunzip)
gunzip file.txt.gz              # decompress (shortcut for gzip
    -d)

gzip -1 file.txt                # fastest compression (low ratio
    )
gzip -9 file.txt                # best compression (slowest)
gzip -v file.txt                # verbose: show compression
    ratio
gzip -l file.txt.gz             # list compressed file info
```

```
gzip -r myfolder/                # compress all files in a
    directory recursively
```

Comparison – zip vs gzip:

Feature	zip	gzip
Multiple files	Yes (bundles them)	No (one file at a time)
Output extension	.zip	.gz
Original deleted	No	Yes (unless -k used)
Cross-platform	Yes (Windows too)	Mainly UNIX/Linux
Paired with tar	No	Yes (`tar -czf`)
Password protect	Yes (`-e`)	No

bzip2 and xz: Other popular compression tools in UNIX. `bzip2` gives better compression than `gzip` but is slower. `xz` gives the best compression. Usage: `bzip2 file` creates `file.bz2`; `xz file` creates `file.xz`.

Q6. Archiving Files in UNIX – `tar`

Archiving means bundling multiple files and directories into a single file for easy storage and transfer. The main archiving tool in UNIX is `tar` (Tape Archive).

Syntax: `tar [options] archive.tar file(s)`

Main tar operations:

Flag	Meaning
`-c`	**Create** a new archive
`-x`	**Extract** files from archive
`-t`	**List** (view) contents of archive
`-r`	**Append** files to an existing archive
`-u`	**Update** – add files newer than copy in archive

Common additional flags:

Flag	Meaning
`-f`	Specifies the archive **filename** (always required)
`-v`	**Verbose** – print files as they are processed
`-z`	Compress/decompress with **gzip** (.tar.gz)
`-j`	Compress/decompress with **bzip2** (.tar.bz2)
`-J`	Compress/decompress with **xz** (.tar.xz)
`-C`	Change to a directory before extracting
`-p`	Preserve file permissions

Creating archives:

```
tar -cvf archive.tar file1 file2     # create archive (no
    compression)
tar -cvf archive.tar myfolder/      # archive an entire
    directory
tar -czvf archive.tar.gz folder/    # create + gzip compress
tar -cjvf archive.tar.bz2 folder/   # create + bzip2 compress
tar -cJvf archive.tar.xz folder/    # create + xz compress
```

Listing archive contents (without extracting):

```
tar -tvf archive.tar                # list all files in archive
tar -tzvf archive.tar.gz            # list contents of gzipped
    archive
```

Extracting archives:

```
tar -xvf archive.tar                # extract to current
    directory
tar -xzvf archive.tar.gz            # extract gzip archive
tar -xjvf archive.tar.bz2           # extract bzip2 archive
tar -xJvf archive.tar.xz            # extract xz archive
tar -xvf archive.tar -C /tmp/       # extract to a specific
    directory
tar -xvf archive.tar file1.txt      # extract only one specific
    file
```

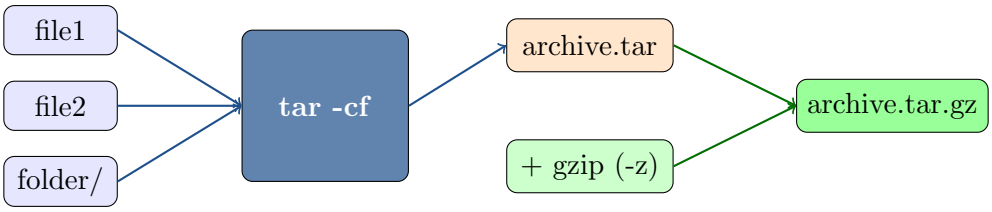
Appending to an existing archive:

```
tar -rvf archive.tar newfile.txt    # add newfile.txt to
    existing archive
tar -uvf archive.tar folder/        # update archive with newer
    files
```

Quick reference – most used tar commands:

Command	What it does
`tar -cvf out.tar folder/`	Create archive
`tar -czvf out.tar.gz folder/`	Create + gzip compress
`tar -tvf out.tar`	List contents
`tar -xvf out.tar`	Extract archive
`tar -xzvf out.tar.gz`	Extract gzip archive
`tar -xvf out.tar -C /dir/`	Extract to specific folder

How tar works with compression:



Full comparison – archive and compression tools:

Tool	Extension	Bundles Files	Notes
tar	.tar	Yes	Archive only, no compression
tar + gzip	.tar.gz	Yes	Most common in Linux
tar + bzip2	.tar.bz2	Yes	Better compression, slower
tar + xz	.tar.xz	Yes	Best compression, slowest
zip	.zip	Yes	Cross-platform, widely used
gzip	.gz	No	Single file compression only
bzip2	.bz2	No	Single file, better than gzip

Memory trick for tar flags:
create **v**erbose **f**ile = `-cvf` (create archive)
e**x**tract **v**erbose **f**ile = `-xvf` (extract archive)
Add **z** in the middle for gzip: `-czvf` / `-xzvf`

— End of Module 4 Notes —